

ReNarrator

Agentic Web Page Re-Narration Engine

Architecture & Pipeline Design Document

Informed by *Agentic Design Patterns* by Antonio Gulli

What is ReNarrator?

ReNarrator is a Chrome extension that transforms any web page to match a user's intent. Through a conversational chat interface, users describe how they want a page re-narrated—simplified for a non-technical audience, rewritten with a marketing lens, condensed into key points, or translated into another language.

Behind the scenes, an **eleven-agent pipeline** powered by Vision-Language Models screenshots the page, understands its visual structure, and rewrites content section by section. A persistent memory system learns each user's preferences over time, and self-improving agents evolve their own instructions based on feedback.

The result: the original page stays untouched on the left, while a fully renarrated version appears in a right panel—with quantifiable metrics showing exactly how readability, length, and clarity improved.

This document describes the complete system architecture, the six-phase agentic pipeline, the memory system, and the design patterns that make it work.



Contents

1	Product Vision	3
1.1	Core Principles	3
1.2	Design Patterns Applied	3
2	System Architecture	4
3	The Agentic Pipeline	5
3.1	Phase 0: Intelligent Routing	5
3.2	Phase 1: Understanding	6
3.2.1	The Three-Layer Memory System	6
3.3	Phase 2: Visual Analysis	7
3.4	Phase 3: Strategic Planning	7
3.5	Phase 4: Parallel Execution	8
3.6	Phase 5: Safety and Validation	8
3.7	Phase 6: Background Learning	9
3.7.1	The Prompt Evolution Engine	9
4	The Dynamic Orchestrator	9
5	Performance and Cost	10
6	Implementation Roadmap	11
7	Security and Privacy	11
8	Conclusion	11

1 Product Vision

ReNarrator sits at the intersection of two ideas: that the same information can serve radically different audiences, and that an AI system can learn *how you think* well enough to perform that translation automatically.

Consider a technical blog post about Kubernetes orchestration. A platform engineer reads it as a reference. A product manager needs it reframed around business impact. A new hire wants it explained from first principles. The text is the same; the *narration* each reader needs is different.

ReNarrator is a Chrome extension that provides this personalized narration layer. The user opens any web page, describes what they need through a conversational chat panel, and the system transforms the page in real time—rewriting text, restructuring sections, and optionally generating diagrams to aid understanding.

1.1 Core Principles

1. **Intent-first.** The system does not just follow commands—it *understands goals* through multi-turn conversation. Saying “I don’t understand this page” is interpreted as a request to simplify vocabulary, add analogies, and include worked examples.
2. **Layout-aware.** Vision-Language Models screenshot the page and understand its visual hierarchy. A `<div>` that looks like a hero banner is treated differently from one that looks like a sidebar callout, regardless of the underlying HTML.
3. **Iterative.** Users refine continuously. “Make the intro punchier” or “now translate it” triggers only the parts of the pipeline that need to re-run, keeping response times under three seconds for follow-up requests.
4. **Non-destructive.** The original page stays untouched on the left; the renarrated version appears in a right panel with synchronized scrolling and section highlighting.
5. **Personalized.** A three-layer memory system learns preferences over time. After ten sessions, ReNarrator already knows your preferred tone, typical audience, and the terminology you like to avoid.

1.2 Design Patterns Applied

This architecture draws on thirteen design patterns from *Agentic Design Patterns* by Antonio Gulli. Each pattern addresses a specific challenge in building reliable multi-agent systems.

Pattern	Ch.	How ReNarrator Uses It
Prompt Chaining	1	Sequential agent pipeline with structured contracts between stages
Routing	2	Pipeline Router selects optimal configuration per request
Parallelization	3	Narrator generates multiple variants for critical sections concurrently
Reflection	4	Quality Validator acts as a Critic with cumulative failure memory
Tool Use	5	VLM uses screenshots; DOM exposed as queryable tool interface
Planning	6	Content Strategist creates adaptive, section-level renarration plans
Multi-Agent	7	Dynamic Orchestrator coordinates eleven specialized agents
Memory	8	Three-layer store: semantic (profile), episodic (sessions), procedural (learned instructions)
Learning	9	Prompt Evolution Engine A/B tests and improves agent prompts over time
Goal Setting	11	Quantifiable readability, length, and jargon-reduction targets
Exception Handling	12	Per-agent fallback strategies with graceful degradation
Human-in-the-Loop	13	Strategic intervention checkpoints when the system is uncertain
Guardrails	18	Factual drift detection, XSS sanitization, bias injection checks

Table 1: Thirteen agentic design patterns and how they are applied in the pipeline.

2 System Architecture

The system is organized into three layers: a Chrome extension that captures page content and renders the renarrated output, a backend server that orchestrates the agentic pipeline, and a persistent memory store that retains user preferences across sessions.

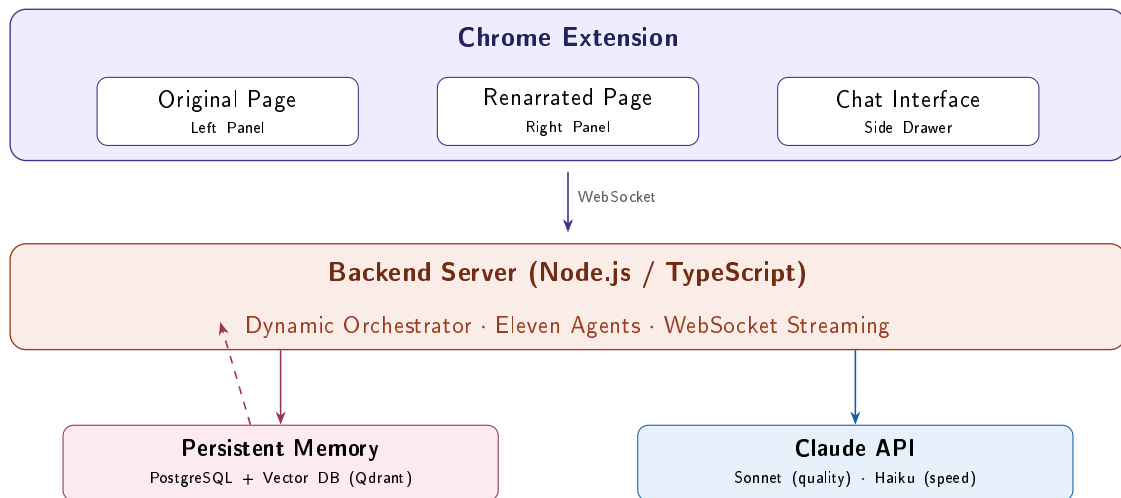


Figure 1: Three-layer system architecture. The extension captures and renders; the backend thinks; the memory persists.

The Chrome extension creates a split-view by constraining the original page to the left half of the viewport and injecting a shadow DOM panel on the right. Scroll positions are synchronized bidirectionally, and hovering over a renarrated section highlights its corresponding original section. The chat interface lives in a collapsible side drawer.

Communication between the extension and backend uses WebSocket connections for real-time streaming. As each section is renarrated, it streams to the extension immediately, so users see the right panel populate progressively rather than waiting for the entire page to be processed.

3 The Agentic Pipeline

The pipeline consists of eleven agents organized into six phases. Each agent is a self-contained unit with a clear input/output contract, a specific LLM model assignment, and defined fallback behavior. The Dynamic Orchestrator—itself backed by an LLM—decides which agents to invoke, in what order, and with which model, based on the specific request.

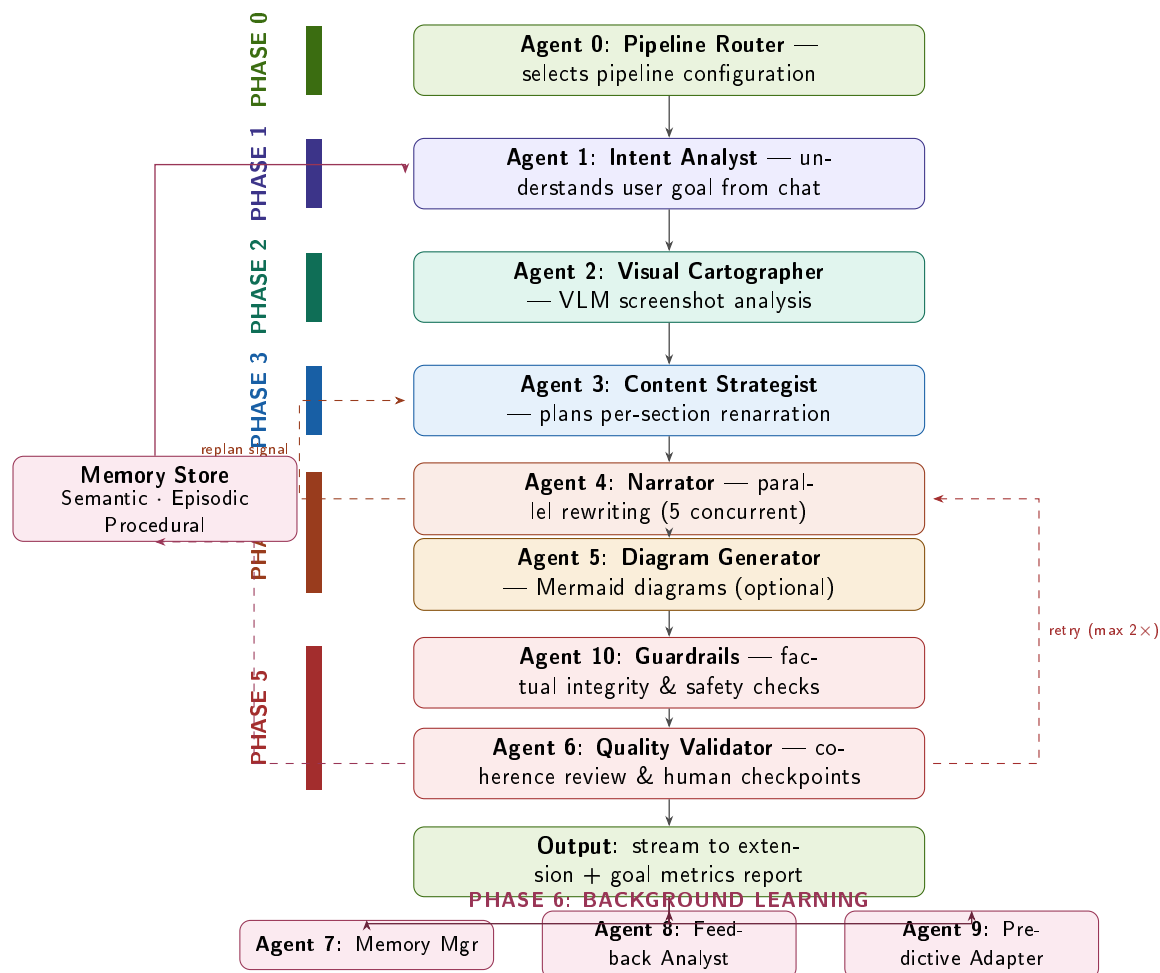


Figure 2: The eleven-agent pipeline across six phases. Solid arrows show the main flow; dashed arrows show the retry loop and adaptive replan signal. Phase 6 agents run asynchronously after each pipeline completion.

The following subsections describe each phase in detail, explaining what each agent does, why it exists as a separate stage, and how it interacts with the agents around it.

3.1 Phase 0: Intelligent Routing

Agent 0 — Pipeline Router Model: Haiku (fast)

Not every page needs the full eleven-agent treatment. A simple translation request on a plain article does not require a VLM screenshot analysis or a content strategist. The Pipeline Router examines the incoming request and page metadata, then selects the optimal pipeline configuration.

For a complex landing page with mixed media, it runs the full pipeline. For a straightforward translation, it routes directly from the Intent Analyst to the Narrator, skipping four agents

entirely. For pages that are already simple, it switches to an annotation mode that adds explanatory tooltips rather than rewriting text. This routing decision alone reduces latency by up to 70% and cost by up to 80% for common operations, without any loss in output quality.

Page Type	Config	What Happens	Est. Time
Complex landing page	Full	All agents run with Sonnet	~10s
Technical article	Lite	Skip VLM (Agent 2); DOM-only analysis	~6s
Foreign language page	Translate	Agent 1 → 4 → 6 only (Haiku)	~3s
Already-simple page	Annotate	Tooltip annotations, no rewriting	~2s

Table 2: Pipeline routing configurations. The Router selects the cheapest path that still produces high-quality output.

3.2 Phase 1: Understanding

Agent 1 — Intent Analyst Model: Sonnet

The Intent Analyst is the system’s conversational brain. It reads the full chat history and produces a structured specification of what the user actually wants—not just what they said. When a user writes “I don’t get this page,” the analyst interprets that as a request to lower the reading level, add explanatory analogies, and include worked examples. When the same user later writes “make the intro punchier,” it produces a minimal delta that targets only the introduction, preserving all other decisions.

The analyst also reads the user’s memory profile. If the user has historically preferred bullet points, concise output, and marketing-oriented language, those preferences are baked into the specification from the start—even if the user didn’t mention them.

To prevent *contextual drift*—where the original user intent gets diluted as it passes through multiple agents—every agent in the pipeline receives a compressed context summary containing the user’s raw message, the pipeline stage number, and key decisions made by prior agents. This ensures that Agent 4, which is three hops from the user’s words, still has direct access to the original intent.

3.2.1 The Three-Layer Memory System

Memory is what transforms ReNarrator from a stateless tool into a personal AI. The system maintains three types of long-term memory, mirroring how human memory works:

- Semantic memory** stores facts about the user: their role (“product manager”), domain expertise levels, preferred output length, terminology they like to avoid, and languages they work in. This is a structured profile stored in PostgreSQL that evolves with confidence-weighted updates.
- Episodic memory** stores past renarration sessions as vector embeddings. When a user opens a Kubernetes documentation page, the system retrieves “last time you opened similar docs, you wanted a non-technical executive summary” and pre-loads that context. Stored in a vector database for semantic similarity search.
- Procedural memory** stores the agents’ own learned instructions. If the Narrator discovers that using passive voice for calls-to-action consistently leads to user edits, that anti-pattern is recorded and the agent’s prompt is dynamically adjusted. This is the most novel layer—agents that learn *how to do their job better* over time.

3.3 Phase 2: Visual Analysis

Agent 2 — Visual Cartographer Model: Sonnet with vision

Raw DOM extraction is blind to visual semantics. A `<div>` might render as a hero banner with a background image, a three-column feature grid, a testimonial card with a portrait photo, or a pricing table with tiered columns. The HTML structure gives no indication of which.

The Visual Cartographer solves this by screenshotting the page and analyzing it with a Vision-Language Model. It identifies each visual section, assigns a semantic role (headline, feature list, call-to-action, code block, navigation), rates its importance, and notes relevant visual context like “large background image with overlay text” or “data table with alternating row colors.”

Rather than receiving the entire DOM upfront—which wastes tokens on irrelevant elements—the agent queries the DOM as a tool interface. It sees something in the screenshot, then asks: “What is the CSS class of that element? How many children does it have? Is it visible?” This on-demand approach is significantly more token-efficient.

If the VLM service fails entirely, the agent falls back to a DOM-only heuristic analysis that uses standard content-detection algorithms. The output quality degrades slightly, but the pipeline does not break.

The Cartographer also performs **smart content detection**, automatically excluding elements that should not be renarrated: navigation bars, cookie consent banners, footers, advertisements, and modal overlays. These are identified through a combination of VLM recognition (“this looks like a cookie banner”) and structural heuristics (elements with `role="navigation"` or common ad-related class names).

3.4 Phase 3: Strategic Planning

Agent 3 — Content Strategist Model: Sonnet

Without a planning stage, each section would be renarrated in isolation, producing a result that feels disjointed. The Content Strategist ensures coherence across the entire page.

It receives the user’s intent and the page map, then produces a section-by-section plan: this headline should be rewritten as a value proposition; these three feature descriptions should lead with business impact rather than technical specs; this pricing section should add ROI framing; this testimonial should be kept as-is because it already supports the narrative.

The plan also establishes consistent terminology. If the first section uses “container orchestration” to describe Kubernetes, every subsequent section uses the same term rather than alternating with “cluster management” or “deployment automation.” This cross-section consistency is something a section-by-section approach without planning cannot achieve.

The Strategist also sets **quantifiable goal metrics** for the renarration. If the user wants simplification, it measures the original page’s Flesch-Kincaid readability score and sets a target. If the user wants condensation, it calculates word count targets per section. These metrics are checked by the Quality Validator and reported to the user in the final output—for example, “Readability improved from grade 14 to grade 8 (target: grade 6).”

When the pipeline operates in iterative mode (the user refining a previous renarration), the Strategist only re-plans the sections affected by the user’s latest instruction, reusing the existing plan for everything else.

The Strategist accepts **adaptive replan signals** from the Narrator. If Agent 4 discovers during writing that a section contains a code block (which the VLM may have missed), it sends a signal back to Agent 3 requesting a revised plan for that section. This prevents the rigid-plan failure mode described in Chapter 6 of the book.

3.5 Phase 4: Parallel Execution

Agent 4 — Narrator Model: Sonnet × 5 concurrent

The Narrator is the writer. It takes each section plan and produces the actual renarrated text. To minimize latency, sections are processed in parallel—up to five concurrent LLM calls, each with the global strategy context to maintain coherence.

For high-importance sections (the hero headline, the primary call-to-action), the Narrator generates two to three variants in parallel. The Quality Validator later selects the best one. This “Best-of-N” approach costs approximately 30% more tokens for critical sections, but produces measurably better output for the parts that matter most.

Results stream back to the extension section by section. The user sees the right panel populate progressively as each section completes, rather than waiting for the entire page to finish.

Agent 5 — Diagram Generator Model: Sonnet (optional)

When complex content benefits from visualization—process flows, system architectures, decision trees, hierarchies—the Diagram Generator creates Mermaid diagrams that are injected into the renarrated panel.

Diagram generation is triggered in two ways: explicitly by the user (“add a diagram of the flow”), or automatically by the Content Strategist when it detects content patterns that would benefit from visual representation. The agent validates all generated Mermaid syntax before sending it to the extension.

3.6 Phase 5: Safety and Validation

Agent 10 — Guardrails Model: Haiku (fast checks)

Before the output reaches the Quality Validator, the Guardrails agent performs four critical safety checks:

Factual integrity. Numbers, dates, proper nouns, and statistical claims are extracted from the original page and verified against the renarrated output. If “\$4.2 billion in annual revenue” becomes “\$42 billion,” or “founded in 2015” shifts to “founded in 2005,” the section is flagged for correction.

XSS sanitization. Renarrated content is sanitized before DOM injection to prevent cross-site scripting attacks from malicious page content that might survive the rewriting process.

Bias detection. The agent checks whether the renarration introduces political, commercial, or cultural bias that was not present in the original. An objective product comparison should not become a one-sided endorsement.

Fabrication detection. Any claims in the renarrated output that do not appear in the original page—even plausible-sounding ones—are flagged. The system renarrates existing content; it should never invent new information.

Agent 6 — Quality Validator Model: Sonnet with vision

The Validator acts as the Critic in a Producer-Critic model. It reviews the complete renarrated output for tone consistency, cross-section coherence, coverage completeness, and alignment with the user’s original intent.

When it identifies issues, it triggers a retry loop: the flagged sections are sent back to the Narrator with the validator’s specific feedback. Critically, the retry loop preserves a **cumulative failure history**. If the first retry attempted to fix a tone issue by increasing formality and that still did not satisfy the validator, the second retry knows not to repeat that approach and tries a different strategy instead. The loop runs at most twice to prevent infinite cycles.

At strategic points, the Validator can pause and involve the user. If the intent confidence is

low, it asks for clarification. If the plan would remove multiple sections, it confirms before proceeding. If the overall quality score falls below a threshold, it offers to try a fundamentally different approach rather than making incremental fixes.

3.7 Phase 6: Background Learning

Three agents run asynchronously after each completed pipeline, updating the memory system and improving future performance.

Agent 7 — Memory Manager Runs: after pipeline completion

Extracts durable preferences from the session and updates all three memory layers. Uses confidence-weighted updates: a single request for “make it simpler” creates a low-confidence episodic memory, while the same request repeated across three different sessions becomes a medium-confidence profile update. An explicit statement like “I always want bullet points” triggers an immediate high-confidence profile change.

Agent 8 — Feedback Analyst Runs: on user feedback/edits

The most novel learning agent. When a user manually edits the renarrated text in the panel, this agent diffs the original renarration against the user’s changes and reverse-engineers what the user actually wanted.

If the user consistently shortens headlines from twelve words to five, the system learns “prefers punchy headlines.” If the user replaces jargon with simpler words, it learns “lower technicality than estimated.” If the user adds a specific phrase, it learns “wants this terminology included.” This implicit learning is more powerful than explicit feedback because users naturally edit text to match their preferences without having to articulate rules.

Agent 9 — Predictive Adapter Runs: on page load (before user types)

Pre-computes likely intents before the user sends any message. When a user opens ReNarrator on a new page, instead of a blank chat, they see personalized suggestions:

“This looks like a Kubernetes documentation page. Based on your history, would you like me to simplify for non-technical stakeholders (like you did last Tuesday), or summarize the key changes for your team?”

The Predictive Adapter makes the system feel proactive rather than reactive. Combined with pre-warmed pipeline state (early screenshot capture, cached page maps for revisited URLs), it can reduce perceived latency by several seconds.

3.7.1 The Prompt Evolution Engine

All three background agents feed into the Prompt Evolution Engine, which implements the self-improvement pattern from Chapter 9. Periodically, the engine generates two to three variant prompts for each agent, A/B tests them against the current production prompt, and promotes the winner. Performance is measured by three metrics: user satisfaction (fewer edits), iteration count (better first-attempt output), and validator pass rate (fewer retries). The evolution history is stored for rollback if a new prompt variant underperforms.

4 The Dynamic Orchestrator

The most transformative architectural decision, inspired by the book’s Hypothesis 5 (Metamorphic Multi-Agent Systems), is making the orchestrator itself an LLM-backed agent.

A static orchestrator runs every agent in sequence for every request. A dynamic orchestrator *reasons about the request first*, then selects which agents to run, with which models, in what order, and whether to parallelize steps.

User Request	Static	Dynamic	Latency	Cost
"Translate this to Turkish"	All 6 phases	Agents 1→4→6 (Haiku)	3s (vs 10s)	\$0.02 (vs \$0.13)
"Make the intro shorter"	All 6 phases	Agent 1→4 (1 section)→6	2s (vs 10s)	\$0.03 (vs \$0.13)
"Full marketing rewrite"	All 6 phases	All 6 phases (Sonnet)	10s	\$0.13
"Add a diagram of the flow"	All 6 phases	Agents 1→5→6	3s (vs 10s)	\$0.03 (vs \$0.13)

Table 3: Static vs. Dynamic Orchestrator. For simple requests, the dynamic approach is 3–5× faster and 4–6× cheaper.

The orchestrator’s reasoning is transparent and logged. For a translation request, it might reason: “This is a translation task on a simple article. The user has done this eight times before with consistent preferences. Skip the Visual Cartographer (no layout analysis needed), skip the Content Strategist (no restructuring), use Haiku for the Narrator (translation is straightforward), and skip the Diagram Generator.” The result is a plan that runs three agents instead of eleven, completing in three seconds instead of ten.

5 Performance and Cost

Agent	Target Latency	Tokens (in/out)	Cost
0. Pipeline Router	<0.5s	500 / 100	\$0.002
1. Intent Analyst	<1s	2,000 / 500	\$0.01
2. Visual Cartographer	<3s	5,000 / 1,000	\$0.02
3. Content Strategist	<2s	3,000 / 1,000	\$0.01
4. Narrator (×5 sections)	<2s	10,000 / 3,000	\$0.04
5. Diagram Generator	<2s	1,500 / 500	\$0.01
10. Guardrails	<1s	3,000 / 300	\$0.01
6. Quality Validator	<2s	5,000 / 500	\$0.02
Full pipeline (first request)	8–12s		~\$0.13
Iterative refinement (2 sections)	2–4s		~\$0.04
Translation (routed)	2–3s		~\$0.02

Table 4: Per-agent performance breakdown. With streaming, users see first results at ~4 seconds even during a full pipeline run.

6 Implementation Roadmap

Phase	Timeline	Deliverables
Foundations	Weeks 1–4	Core pipeline with prompt chaining, exception handling with fallback strategies, Guardrails agent, Pipeline Router for cost optimization
Intelligence	Weeks 5–8	Three-layer memory system, reflection with failure memory, goal metrics engine, inter-agent signal bus
Evolution	Weeks 9–12	Prompt Evolution Engine, Dynamic Orchestrator, human-in-the-loop checkpoints, Best-of-N generation for critical sections
Polish	Weeks 13–16	Adaptive replanning, DOM tool interface, performance benchmarking, user-facing memory dashboard

Table 5: Sixteen-week implementation roadmap organized by capability maturity.

7 Security and Privacy

- **Ephemeral page data.** Screenshots and DOM content are stored with a one-hour time-to-live. No raw page content is retained permanently.
- **Encrypted credentials.** User API keys are encrypted at rest and never appear in logs or error reports.
- **Content sanitization.** All renarrated content is sanitized before DOM injection. The Guardrails agent provides an additional safety layer against XSS and injection attacks.
- **Rate limiting.** Per-user rate limits prevent abuse of the pipeline.
- **Memory transparency.** Users have full control over their memory through a dashboard: view everything ReNarrator remembers, selectively delete memories, export their profile as JSON, or pause memory entirely for an incognito-like experience. Episodic memories auto-expire after 90 days unless explicitly pinned.

8 Conclusion

ReNarrator treats web page renarration as a *multi-agent editorial process*—not a single prompt. Eleven specialized agents, organized across six phases and informed by thirteen agentic design patterns, produce output that is coherent, factually sound, and tailored to each user’s needs.

Three innovations distinguish this architecture:

1. **The Dynamic Orchestrator** plans the pipeline itself, choosing which agents to run based on the specific request. Simple tasks skip unnecessary agents, reducing cost by up to 80% with no quality loss.
2. **The three-layer memory system** transforms a stateless tool into a personal AI. Semantic memory stores who you are, episodic memory stores what you’ve done, and procedural memory teaches agents how to serve you better.
3. **Self-improving agents** evolve their own prompts through A/B testing and implicit learning from user edits. The system gets measurably better with every interaction—not through retraining, but through the intelligent management of context.

The result is a system that feels less like a tool and more like a knowledgeable colleague: one that understands what you need, remembers how you like it, and gets better at helping you every time.